Parking Management System

Database Management Systems Project

Compulsory Interview Questions

Theory, Queries & Concepts

Covers: ER Diagrams, Normalization, SQL (DDL/DML/DRL/TCL/DCL), Joins, Subqueries, Views, Triggers, Stored Procedures, Transactions, Window Functions & More

June 11, 2026

Contents

1 Project Overview Questions

Q1: What is the Parking Management System?

The Parking Management System is a relational database project that manages parking slots, users, vehicles, reservations, parking records, and payments across multiple parking facilities. It is designed for efficient parking and slot allocation using MySQL.
Key entities: Facility, Parking_Slot, User, Vehicle, Reservation, Parking_Record, Payment, Staff.
Core features:

- Multi-facility support with floor-wise slot management
- Vehicle registration and user membership tiers (Basic/Premium/VIP)
- Advance reservation and real-time parking tracking
- Automated billing based on duration and hourly rate
- Staff management per facility

Q2: What tools and technologies did you use?

- **DBMS:** MySQL 8.x (via MySQL Workbench)
- **SQL Features:** DDL, DML, DRL, TCL, DCL, Stored Procedures, Triggers, Views, Window Functions, Transactions
- **Design:** ER Diagrams, Normalization to BCNF
- **Schema:** 8 tables with proper foreign key constraints, indexes, and referential integrity

Q3: How many tables are in your schema? List them and their purpose.

Table	Purpose
Facility	Stores parking facility/garage details (name, address, floors)
Parking_Slot	Individual slots within a facility (type, rate, occupancy status)
User	Registered users of the system with membership tier
Vehicle	Vehicles owned by users (type, brand, license plate)
Reservation	Advance slot bookings with time range and status
Parking_Record	Actual entry/exit log with computed duration
Payment	Payment for each parking session (amount, method, status)
Staff	Employees managing each facility (role, salary)

2 ER Diagram Questions

Q4: Explain the ER diagram of your project.

The ER diagram consists of 8 entities with the following relationships:

One-to-Many (1:M) Relationships:

- Facility → Parking_Slot (a facility has many slots)
- Facility → Staff (a facility employs many staff)
- User → Vehicle (a user owns many vehicles)
- User → Reservation (a user makes many reservations)
- User → Payment (a user makes many payments)
- Vehicle → Parking_Record (a vehicle has many parking sessions)
- Vehicle → Reservation (a vehicle has many reservations)
- Parking_Slot → Parking_Record (a slot is used many times)
- Parking_Slot → Reservation (a slot is reserved many times)

One-to-One (1:1) Relationship:

- Parking_Record ↔ Payment (each record has exactly one payment)

Primary Keys: Each entity has a surrogate key (`*_id` with `AUTO_INCREMENT`).

Candidate Keys: `User.email`, `User.license_no`, `Vehicle.license_plate`, `Parking_Slot(facility_id, floor_number, slot_number)`.

Q5: What are the different types of attributes in your ER model?

- **Simple:** `first_name`, `email`, `phone`
- **Composite:** Full name = `first_name` + `last_name`; Address = `address` + `city` + `state` + `zip_code`
- **Derived:** `duration_hours` (computed from `entry_time` and `exit_time`); `estimated_bill` (computed in views)
- **Multi-valued:** Not present (1NF compliant); a user's multiple vehicles are stored in a separate Vehicle table
- **Key:** `facility_id`, `slot_id`, `user_id`, etc.

Q6: What is cardinality? Explain with examples from your project.

Cardinality defines the number of instances of one entity that can be associated with instances of another entity.

- **1:1** — Parking_Record to Payment. Each parking session generates exactly one payment, and each payment belongs to exactly one record. Enforced via `UNIQUE` constraint on `Payment.record_id`.
- **1:M** — User to Vehicle. One user can own multiple vehicles, but each vehicle belongs to one user. Enforced via `Vehicle.user_id` foreign key.
- **M:N** — Not directly present. However, Users and Slots have an implicit M:N relationship resolved through the Reservation and Parking_Record junction tables.

3 Normalization Questions

Q7: What is normalization? Why is it important?

Normalization is the process of organizing a relational database to reduce data redundancy and eliminate undesirable anomalies (insertion, update, deletion anomalies).
Importance:

- Eliminates redundant data storage
- Prevents insertion anomalies (can't add data without unrelated data)
- Prevents update anomalies (changing data in one place but not another)
- Prevents deletion anomalies (deleting data accidentally removes other data)
- Ensures data integrity and consistency

Q8: Show that your schema is in 1NF, 2NF, 3NF, and BCNF.

1NF (First Normal Form):
- All attributes are atomic (no multi-valued attributes)
- Every table has a primary key
- No repeating groups (e.g., multiple vehicles stored in separate rows, not in one column)

2NF (Second Normal Form):
- Already in 1NF
- No partial dependencies: all primary keys are single-column (`AUTO_INCREMENT`), so partial dependency is impossible

3NF (Third Normal Form):
- Already in 2NF
- No transitive dependencies: no non-key attribute determines another non-key attribute
- Example: `slot_type` does NOT determine `hourly_rate` because different facilities can charge different rates for the same slot type

BCNF (Boyce-Codd Normal Form):
- Already in 3NF
- For every functional dependency $X \rightarrow Y$, X is a superkey
- Example: In `User`, `user_id` $\rightarrow$ all attributes (superkey); `email` $\rightarrow$ `user_id` (email is a candidate key, hence a superkey)

Q9: List all functional dependencies in your schema.

```
Facility:
  facility_id -> name, address, city, state, zip_code,
     total_floors

Parking_Slot:
  slot_id -> facility_id, floor_number, slot_number, slot_type,
             is_occupied, hourly_rate
  (facility_id, floor_number, slot_number) -> slot_id

User:
  user_id    -> first_name, last_name, email, phone, license_no,
     membership
  email      -> user_id
  license_no -> user_id

Vehicle:
  vehicle_id    -> user_id, license_plate, vehicle_type, brand,
     model, color
  license_plate -> vehicle_id

Reservation:
  reservation_id -> user_id, slot_id, vehicle_id, reserved_from,
                    reserved_until, status

Parking_Record:
  record_id -> vehicle_id, slot_id, entry_time, exit_time,
     duration_hours

Payment:
  payment_id -> record_id, user_id, amount, payment_method,
                payment_status, paid_at
  record_id  -> payment_id

Staff:
  staff_id -> facility_id, first_name, last_name, role, phone,
              salary, hire_date
```

In every case, the determinant is a superkey ⇒ BCNF satisfied.

Q10: What is the difference between 3NF and BCNF?

- **3NF:** For every FD $X \to A$, either X is a superkey OR A is a prime attribute (part of a candidate key).
- **BCNF:** For every FD $X \to A$, X **must** be a superkey. No exception for prime attributes.
- BCNF is stricter than 3NF. A relation in BCNF is always in 3NF, but not vice versa.
- **Example of difference:** Consider a table `(Student, Course, Instructor)` where each instructor teaches only one course. FD: `Instructor` → `Course`. Here `Instructor` is not a superkey, but `Course` is a prime attribute (part of candidate key). This satisfies 3NF but violates BCNF.
- In our project, since all determinants are superkeys, both 3NF and BCNF are satisfied.

4 SQL Theory Questions

Q11: What are the different types of SQL commands? Give examples from your project.

Category	Commands	Project Example
DDL (Data Definition)	CREATE, ALTER, DROP	`CREATE TABLE Facility (.`
DML (Data Manipulation)	INSERT, UPDATE, DELETE	`INSERT INTO User VALUES`
DRL/DQL (Data Retrieval)	SELECT	`SELECT * FROM vw_facility_occupancy`
TCL (Transaction Control)	COMMIT, ROLLBACK, SAVEPOINT	`START TRANSACTION; ... C`
DCL (Data Control)	GRANT, REVOKE	`GRANT SELECT ON parking_management.*`

Q12: What is the difference between DELETE, TRUNCATE, and DROP?

	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
What it removes	Specific rows (with WHERE)	All rows	Entire table (structure + data)
Rollback	Yes (within transaction)	No	No
WHERE clause	Supported	Not supported	Not applicable
Triggers fired	Yes	No	No
Space reclaim	No (until OPTIMIZE)	Yes	Yes

Project example:

```
-- DELETE specific rows
DELETE FROM Reservation WHERE status = 'Cancelled';

-- TRUNCATE all rows
TRUNCATE TABLE Parking_Record;

-- DROP entire table
DROP TABLE Staff;
```

Q13: What are constraints? Which ones did you use?

Constraints enforce rules on data to maintain integrity.

Constraint	Purpose	Project Usage
PRIMARY KEY	Uniquely identifies each row	`user_id INT AUTO_INCREMENT PRIMARY KEY`
FOREIGN KEY	Enforces referential integrity	`FOREIGN KEY (facility_id) REFERENCES Facility(facility_id)`
UNIQUE	No duplicate values	`email VARCHAR(100) UNIQUE`
NOT NULL	Column cannot be NULL	`name VARCHAR(100) NOT NULL`
DEFAULT	Sets default value	`is_occupied BOOLEAN DEFAULT FALSE`
CHECK (via ENUM)	Restricts allowed values	`slot_type ENUM('Compact','Regular','Large',...)`

Q14: What is referential integrity? How is it enforced in your project?

Referential integrity ensures that a foreign key value in a child table always refers to a valid primary key in the parent table.
How enforced:

```
FOREIGN KEY (facility_id) REFERENCES Facility(facility_id)
    ON DELETE CASCADE ON UPDATE CASCADE
```

- **ON DELETE CASCADE:** If a facility is deleted, all its slots, records, and payments are automatically deleted
- **ON UPDATE CASCADE:** If a facility_id changes, all referencing rows are automatically updated
- Prevents orphan records (e.g., a vehicle referencing a non-existent user)

5 Keys & Indexing Questions

Q15: Explain all types of keys with examples from your project.

- **Super Key:** Any set of attributes that uniquely identifies a row.
  Example: `{user_id}`, `{user_id, email}`, `{email}` in User table.
- **Candidate Key:** Minimal super key (no proper subset is a super key).
  Example: `user_id`, `email`, `license_no` in User table.
- **Primary Key:** The chosen candidate key for the table.
  Example: `user_id` in User table.
- **Alternate Key:** Candidate keys not chosen as primary key.
  Example: `email` and `license_no` in User table.
- **Foreign Key:** An attribute referencing the primary key of another table.
  Example: `Vehicle.user_id` references `User.user_id`.
- **Composite Key:** A key with multiple attributes.
  Example: `(facility_id, floor_number, slot_number)` is a composite candidate key in Parking_Slot.

Q16: What are indexes? Which indexes did you create and why?

An **index** is a data structure (typically B-Tree in MySQL) that speeds up data retrieval at the cost of additional storage and slower writes.

Indexes in the project:

```
CREATE INDEX idx_slot_facility ON Parking_Slot(facility_id,
   is_occupied);
CREATE INDEX idx_vehicle_user  ON Vehicle(user_id);
CREATE INDEX idx_record_entry  ON Parking_Record(entry_time);
CREATE INDEX idx_payment_status ON Payment(payment_status);
```

Why these indexes:

- `idx_slot_facility`: Frequently query available slots per facility
- `idx_vehicle_user`: Join Vehicle with User on every parking query
- `idx_record_entry`: Range queries on parking times (reports, analytics)
- `idx_payment_status`: Filter completed vs. pending payments

MySQL also automatically creates indexes on PRIMARY KEY and UNIQUE columns.

6 Joins Questions

Q17: Explain all types of JOINs with SQL examples from your project.

1. INNER JOIN — Returns rows with matching values in both tables.

```
-- Users who have made payments
SELECT u.first_name, p.amount
FROM User u
INNER JOIN Payment p ON u.user_id = p.user_id;
```

2. LEFT (OUTER) JOIN — All rows from left table + matching from right.

```
-- All users, even those without reservations
SELECT u.first_name, r.reservation_id
FROM User u
LEFT JOIN Reservation r ON u.user_id = r.user_id;
```

3. RIGHT (OUTER) JOIN — All rows from right table + matching from left.

```
-- All slots, even those never parked in
SELECT ps.slot_number, pr.record_id
FROM Parking_Record pr
RIGHT JOIN Parking_Slot ps ON pr.slot_id = ps.slot_id;
```

4. CROSS JOIN — Cartesian product of both tables.

```
-- All user-facility combinations
SELECT u.first_name, f.name
FROM User u CROSS JOIN Facility f;
```

5. SELF JOIN — A table joined with itself.

```
-- Staff members at the same facility
SELECT s1.first_name AS staff1, s2.first_name AS staff2
FROM Staff s1
JOIN Staff s2 ON s1.facility_id = s2.facility_id
  AND s1.staff_id < s2.staff_id;
```

Q18: What is a natural join? How is it different from an inner join?

- **Natural Join:** Automatically joins on columns with the **same name** in both tables. No ON clause needed.
- **Inner Join:** Requires an explicit ON clause specifying the join condition.
- **Danger of Natural Join:** If two tables share a column name by coincidence (e.g., `phone` in both User and Staff), it joins on that too, producing incorrect results.
- **Best practice:** Always use explicit INNER JOIN with ON clause.

```
-- Natural join (not recommended)
SELECT * FROM Vehicle NATURAL JOIN User;

-- Equivalent explicit inner join (recommended)
SELECT * FROM Vehicle v INNER JOIN User u ON v.user_id = u.user_id
    ;
```

7 Subqueries & Nested Queries

Q19: What are subqueries? Explain with examples.

A **subquery** (nested query) is a SELECT statement inside another SQL statement.
Types and examples:
1. Scalar Subquery (returns single value):

```
-- Users who spent more than average
SELECT first_name, amount FROM Payment p
JOIN User u ON p.user_id = u.user_id
WHERE amount > (SELECT AVG(amount) FROM Payment);
```

2. Row Subquery (returns single row):

```
-- Facility with highest occupancy
SELECT name FROM Facility WHERE facility_id = (
    SELECT facility_id FROM Parking_Slot
    GROUP BY facility_id ORDER BY SUM(is_occupied) DESC LIMIT 1
);
```

3. Table Subquery (returns a table):

```
-- Used in FROM clause
SELECT facility, occupancy_pct FROM (
    SELECT f.name AS facility,
           SUM(s.is_occupied)*100.0/COUNT(*) AS occupancy_pct
    FROM Parking_Slot s JOIN Facility f ON s.facility_id = f.
       facility_id
    GROUP BY f.facility_id, f.name
) AS sub WHERE occupancy_pct > 30;
```

4. Correlated Subquery (references outer query):

```
-- Users whose total spend exceeds 500
SELECT first_name, (
    SELECT SUM(amount) FROM Payment p
    WHERE p.user_id = u.user_id
) AS total_spent
FROM User u HAVING total_spent > 500;
```

Q20: What is the difference between IN, EXISTS, and ANY/ALL?

- **IN:** Checks if a value matches any value in a list or subquery result.
- **EXISTS:** Returns TRUE if the subquery returns at least one row. More efficient for correlated subqueries.
- **ANY/SOME:** Compares a value with any single value from the subquery using an operator (>, <, =).
- **ALL:** Compares a value with every value from the subquery.

```
-- IN: Facilities with EV slots
SELECT name FROM Facility WHERE facility_id IN (
    SELECT facility_id FROM Parking_Slot WHERE slot_type = 'EV');

-- EXISTS: Same query, more efficient
SELECT name FROM Facility f WHERE EXISTS (
    SELECT 1 FROM Parking_Slot ps
    WHERE ps.facility_id = f.facility_id AND ps.slot_type = 'EV');

-- ANY: Users who paid more than ANY VIP user's payment
SELECT * FROM Payment WHERE amount > ANY (
    SELECT amount FROM Payment p JOIN User u ON p.user_id = u.
       user_id
    WHERE u.membership = 'VIP');

-- ALL: Payments greater than ALL basic user payments
SELECT * FROM Payment WHERE amount > ALL (
    SELECT amount FROM Payment p JOIN User u ON p.user_id = u.
       user_id
    WHERE u.membership = 'Basic');
```

8 Aggregate Functions & GROUP BY

Q21: What are aggregate functions? List all with examples.

Aggregate functions perform calculations on a set of values and return a single result.

Function	Purpose	Project Example
COUNT()	Count rows	`SELECT COUNT(*) FROM Parking_Slot WHERE is_occupied = TRUE;`
SUM()	Sum of values	`SELECT SUM(amount) FROM Payment WHERE payment_status = 'Completed';`
AVG()	Average value	`SELECT AVG(duration_hours) FROM Parking_Record;`
MAX()	Maximum value	`SELECT MAX(salary) FROM Staff;`
MIN()	Minimum value	`SELECT MIN(hourly_rate) FROM Parking_Slot;`
GROUP_CONCAT()	Concatenate values	`SELECT GROUP_CONCAT(license_plate) FROM Vehicle GROUP BY user_id;`

Q22: What is the difference between WHERE and HAVING?

	WHERE	HAVING
Applied	Before GROUP BY (filters rows)	After GROUP BY (filters groups)
Aggregates	Cannot use aggregate functions	Can use aggregate functions
Usage	Filter individual rows	Filter grouped results

```sql
-- WHERE: Filter rows before grouping
SELECT facility_id, COUNT(*) AS cnt FROM Parking_Slot
WHERE slot_type = 'Regular'  -- filters individual rows
GROUP BY facility_id;

-- HAVING: Filter groups after grouping
SELECT facility_id, COUNT(*) AS cnt FROM Parking_Slot
GROUP BY facility_id
HAVING cnt > 5;  -- filters entire groups
```

9 Views Questions

Q23: What are views? What views did you create?

A **view** is a virtual table based on a SELECT query. It does not store data; it dynamically executes the query when accessed.

Views in the project:

1. `vw_facility_occupancy` — Dashboard showing total, occupied, and available slots per facility with occupancy percentage.
2. `vw_revenue_summary` — Revenue breakdown per facility (total transactions, revenue, average).
3. `vw_active_sessions` — Currently parked vehicles with estimated bill based on time parked.

```
CREATE OR REPLACE VIEW vw_facility_occupancy AS
SELECT f.name AS facility,
       COUNT(s.slot_id) AS total_slots,
       SUM(s.is_occupied) AS occupied_slots,
       ROUND(SUM(s.is_occupied)*100.0/COUNT(s.slot_id), 1) AS
          occupancy_pct
FROM Facility f
JOIN Parking_Slot s ON f.facility_id = s.facility_id
GROUP BY f.facility_id, f.name;
```

Advantages: Simplifies complex queries, provides data abstraction, enhances security (expose views instead of base tables).

10 Stored Procedures & Functions

Q24: What stored procedures did you create? Explain one in detail.

Three stored procedures:

1. `sp_park_vehicle(vehicle_id, slot_id)` — Parks a vehicle
2. `sp_exit_vehicle(record_id, payment_method)` — Exits vehicle and generates bill
3. `sp_facility_report(facility_id)` — Comprehensive facility report

Detailed explanation of `sp_exit_vehicle`:

```
DELIMITER //
CREATE PROCEDURE sp_exit_vehicle(
    IN p_record_id INT, IN p_payment_method VARCHAR(20))
BEGIN
    -- 1. Set exit_time to NOW()
    UPDATE Parking_Record SET exit_time = NOW()
    WHERE record_id = p_record_id;

    -- 2. Calculate duration and amount
    -- amount = hours * hourly_rate (minimum 1 hour charge)

    -- 3. Insert payment record

    -- 4. Free the slot (set is_occupied = FALSE)

    -- 5. Return bill summary
END //
DELIMITER ;
```

This procedure atomically handles: updating the record, calculating the bill, recording the payment, and freeing the slot — all in a single call.

Q25: Difference between stored procedure and function?

	Stored Procedure	**Function**
Return value	0 or more via OUT params	Exactly one via RETURN
Called with	`CALL sp_name()`	Used in SELECT expressions
DML allowed	Yes (INSERT, UPDATE, DELETE)	Limited (no permanent changes in MySQL)
Transaction	Can use COMMIT/ROLLBACK	Cannot
Use in SQL	Cannot be used in SELECT/WHERE	Can be used in SELECT/WHERE

11 Triggers Questions

Q26: What are triggers? Explain the triggers in your project.

A **trigger** is a stored program that automatically executes in response to an event (INSERT, UPDATE, DELETE) on a table.
Triggers in the project:
1. trg_after_park (AFTER INSERT on Parking_Record):

```
-- Automatically marks the slot as occupied when a vehicle parks
CREATE TRIGGER trg_after_park AFTER INSERT ON Parking_Record
FOR EACH ROW
BEGIN
    UPDATE Parking_Slot SET is_occupied = TRUE
    WHERE slot_id = NEW.slot_id;
END;
```

2. trg_after_exit (AFTER UPDATE on Parking_Record):

```
-- Frees the slot when exit_time is set
CREATE TRIGGER trg_after_exit AFTER UPDATE ON Parking_Record
FOR EACH ROW
BEGIN
    IF OLD.exit_time IS NULL AND NEW.exit_time IS NOT NULL THEN
        UPDATE Parking_Slot SET is_occupied = FALSE
        WHERE slot_id = NEW.slot_id;
    END IF;
END;
```

3. trg_before_user_delete (BEFORE DELETE on User): Prevents deleting a user who has vehicles currently parked.
Types: BEFORE vs AFTER × INSERT/UPDATE/DELETE = 6 types possible.

12 Transaction & Concurrency Questions

Q27: What are ACID properties? How does your project ensure them?

- **Atomicity:** A transaction is all-or-nothing. If vehicle exit fails mid-way, the entire operation rolls back.
  Project: `sp_exit_vehicle` updates record, inserts payment, and frees slot atomically.
- **Consistency:** Database moves from one valid state to another. Foreign keys, UNIQUE constraints, and triggers ensure no invalid data.
  Project: `trg_before_user_delete` prevents orphaning active parking sessions.
- **Isolation:** Concurrent transactions don't interfere. MySQL's InnoDB engine uses row-level locking.
  Project: Two users trying to book the same slot — only one succeeds.
- **Durability:** Once committed, data survives crashes. InnoDB uses write-ahead logging (redo log).
  Project: After `COMMIT`, payment records persist even if server restarts.

Q28: Explain COMMIT, ROLLBACK, and SAVEPOINT with examples.

```
-- COMMIT: Make changes permanent
START TRANSACTION;
    UPDATE Parking_Slot SET is_occupied = FALSE WHERE slot_id = 2;
    UPDATE Parking_Slot SET is_occupied = TRUE  WHERE slot_id = 3;
COMMIT;  -- Both changes saved permanently

-- ROLLBACK: Undo all changes since START TRANSACTION
START TRANSACTION;
    DELETE FROM User WHERE user_id = 5;
ROLLBACK;  -- User 5 is NOT deleted

-- SAVEPOINT: Partial rollback
START TRANSACTION;
    UPDATE User SET membership = 'VIP' WHERE user_id = 7;
    SAVEPOINT before_delete;
    DELETE FROM Reservation WHERE reservation_id = 13;
    ROLLBACK TO before_delete;  -- Reservation NOT deleted
COMMIT;  -- Membership change IS saved
```

13 Window Functions

Q29: What are window functions? Give examples from your project.

Window functions perform calculations across a set of rows related to the current row, without collapsing them into a single output row (unlike GROUP BY).
Syntax: `function() OVER (PARTITION BY ... ORDER BY ...)`
Examples from the project:
1. RANK(): Rank facilities by revenue.

```
SELECT f.name, SUM(p.amount) AS revenue,
       RANK() OVER (ORDER BY SUM(p.amount) DESC) AS revenue_rank
FROM Payment p
JOIN Parking_Record pr ON p.record_id = pr.record_id
JOIN Parking_Slot ps ON pr.slot_id = ps.slot_id
JOIN Facility f ON ps.facility_id = f.facility_id
GROUP BY f.facility_id, f.name;
```

2. Running total:

```
SELECT payment_id, paid_at, amount,
       SUM(amount) OVER (ORDER BY paid_at) AS running_total
FROM Payment WHERE payment_status = 'Completed';
```

3. DENSE_RANK(): Rank users by spending.

```
SELECT CONCAT(u.first_name, ' ', u.last_name) AS customer,
       SUM(p.amount) AS total_spent,
       DENSE_RANK() OVER (ORDER BY SUM(p.amount) DESC) AS rank
FROM Payment p JOIN User u ON p.user_id = u.user_id
GROUP BY u.user_id;
```

Difference: `RANK()` skips numbers after ties (1,2,2,4); `DENSE_RANK()` does not (1,2,2,3).

14 Advanced Theory Questions

Q30: What is denormalization? Is there any in your project?

Denormalization is the intentional introduction of redundancy to improve read performance.
In our project: The `Payment` table stores `user_id` directly, even though it could be derived via `Payment` → `Parking_Record` → `Vehicle` → `User`. This avoids a 3-table join for every payment query, trading minimal redundancy for significant query performance.
When to denormalize:

- Read-heavy workloads where join cost is high
- Reporting/analytics queries that run frequently
- When the redundant data rarely changes

Q31: What is the difference between ENUM and a lookup table?

Our project uses `ENUM` for fixed-value columns like `slot_type`, `membership`, and `payment_method`.

	ENUM	Lookup Table
Storage	Stored as integer internally (efficient)	Requires JOIN (extra table)
Modification	Requires ALTER TABLE to add values	Simple INSERT into lookup table
Best for	Small, rarely-changing value sets	Frequently changing or large value sets
Portability	MySQL-specific	Standard SQL (works everywhere)

We chose ENUM because slot types, membership tiers, and payment methods are stable, small sets that rarely change.

Q32: Explain the `GENERATED ALWAYS AS` column in your schema.

In `Parking_Record`, the `duration_hours` column is a **generated (computed) column**:

```
duration_hours DECIMAL(5,2) GENERATED ALWAYS AS (
    CASE WHEN exit_time IS NOT NULL
         THEN TIMESTAMPDIFF(MINUTE, entry_time, exit_time) / 60.0
         ELSE NULL
    END
) STORED;
```

- **GENERATED ALWAYS:** Value is automatically computed; cannot be manually set
- **STORED:** Value is computed on INSERT/UPDATE and physically stored (vs. VIRTUAL which computes on read)
- **Advantage:** No application logic needed; consistent duration calculation; can be indexed
- **Trade-off:** Uses extra storage, but avoids repeated computation on reads

Q33: What is an ER model? Explain the components.

The **Entity-Relationship (ER) Model** is a conceptual data model used to describe the structure of a database.
Components:

- **Entity:** A real-world object (rectangle). Example: User, Vehicle, Facility.
- **Attribute:** Property of an entity (oval). Example: `first_name`, `email`.
- **Relationship:** Association between entities (diamond). Example: User *OWNS* Vehicle.
- **Primary Key:** Underlined attribute. Example: `user_id`.
- **Cardinality:** 1:1, 1:M, M:N notation on relationship lines.
- **Participation:** Total (double line, entity must participate) vs. Partial (single line, optional).

In our project, User has **total participation** in Vehicle (every user must register a vehicle), while Vehicle has **partial participation** in Parking_Record (a vehicle may or may not have parked).

Q34: What is the difference between CHAR and VARCHAR?

	CHAR(n)	**VARCHAR(n)**
Storage	Fixed-length (always n bytes)	Variable-length (actual length + 1-2 bytes)
Padding	Right-padded with spaces	No padding
Performance	Faster for fixed-size data	Slightly slower (length prefix overhead)
Use case	Fixed-length codes (state codes, zip)	Variable-length strings (names, emails)

In our project: `zip_code VARCHAR(10)` (varies by country), `email VARCHAR(100)` (variable length).
For fixed-format data like a 2-letter state code, `CHAR(2)` would be more efficient.

Q35: What are the different isolation levels in MySQL?

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
READ UNCOMMITTED	Yes	Yes	Yes	Fastest
READ COMMITTED	No	Yes	Yes	Fast
REPEATABLE READ (MySQL default)	No	No	Yes	Moderate
SERIALIZABLE	No	No	No	Slowest

In our project context:

- When two users try to reserve the same slot, REPEATABLE READ prevents one from seeing the other's uncommitted reservation
- SERIALIZABLE would be needed if we must guarantee no phantom slots appear during a range query

15 Practical Query Questions

Q36: Write a query to find all available EV slots across all facilities.

```
SELECT f.name AS facility, ps.floor_number, ps.slot_number, ps.
   hourly_rate
FROM Parking_Slot ps
JOIN Facility f ON ps.facility_id = f.facility_id
WHERE ps.slot_type = 'EV' AND ps.is_occupied = FALSE
ORDER BY f.name, ps.floor_number;
```

Q37: Write a query for the top 3 highest-revenue facilities.

```
SELECT f.name AS facility, SUM(p.amount) AS total_revenue
FROM Payment p
JOIN Parking_Record pr ON p.record_id = pr.record_id
JOIN Parking_Slot ps ON pr.slot_id = ps.slot_id
JOIN Facility f ON ps.facility_id = f.facility_id
WHERE p.payment_status = 'Completed'
GROUP BY f.facility_id, f.name
ORDER BY total_revenue DESC
LIMIT 3;
```

Q38: Write a query to find peak parking hours.

```sql
SELECT HOUR(entry_time) AS hour_of_day,
       COUNT(*) AS total_entries
FROM Parking_Record
GROUP BY HOUR(entry_time)
ORDER BY total_entries DESC;
```

This helps management schedule more staff during peak hours.

Q39: Write a query to simulate membership-based discounts.

```sql
SELECT CONCAT(u.first_name, ' ', u.last_name) AS customer,
       u.membership, p.amount AS original,
       CASE u.membership
           WHEN 'VIP'     THEN ROUND(p.amount * 0.80, 2)  -- 20%
              off
           WHEN 'Premium' THEN ROUND(p.amount * 0.90, 2)  -- 10%
              off
           ELSE p.amount                                   -- no
              discount
       END AS discounted_amount
FROM Payment p
JOIN User u ON p.user_id = u.user_id
WHERE p.payment_status = 'Completed';
```

Q40: Write a query using GROUP_CONCAT to list all vehicles per user.

```sql
SELECT CONCAT(u.first_name, ' ', u.last_name) AS owner,
       COUNT(v.vehicle_id) AS vehicle_count,
       GROUP_CONCAT(v.license_plate ORDER BY v.license_plate
                    SEPARATOR ', ') AS all_plates
FROM User u
JOIN Vehicle v ON u.user_id = v.user_id
GROUP BY u.user_id
ORDER BY vehicle_count DESC;
```

16 Miscellaneous Interview Questions

Q41: What challenges did you face in this project?

- **Concurrent slot booking:** Two users booking the same slot simultaneously. Solved with UNIQUE constraints and transaction isolation.
- **Duration calculation:** Computing parking duration dynamically. Solved with MySQL's `GENERATED ALWAYS AS` stored computed column.
- **Cascading deletes:** Ensuring referential integrity when deleting facilities or users. Solved with `ON DELETE CASCADE` and protective triggers.
- **Billing edge cases:** Minimum charge enforcement (at least 1 hour). Handled in the stored procedure logic.
- **Normalization vs. Performance:** Decided to store `user_id` in Payment (slight denormalization) to avoid expensive 3-table joins for common payment queries.

Q42: How would you scale this system for a real-world deployment?

- **Partitioning:** Partition `Parking_Record` by month (range partitioning on `entry_time`) for faster historical queries.
- **Replication:** Master-slave replication — writes go to master, reads from replicas for load balancing.
- **Caching:** Use Redis to cache available slot counts (frequently queried, rarely changed).
- **Sharding:** Shard by `facility_id` if scaling across geographies.
- **Connection pooling:** Use a connection pool to handle concurrent users efficiently.
- **API layer:** Add a REST API (Node.js / Spring Boot) between the frontend and database.
- **Monitoring:** Add query performance monitoring with MySQL's slow query log.

Q43: What is the difference between clustered and non-clustered indexes?

	Clustered Index	**Non-Clustered Index**
Data order	Physically reorders table data	Separate structure with pointers to data
Count per table	Only 1 (usually the PRIMARY KEY)	Multiple allowed
Speed	Faster for range queries	Faster for specific lookups
Storage	No extra storage (data IS the index)	Additional storage required

In MySQL InnoDB: PRIMARY KEY = clustered index. Our additional indexes (`idx_slot_facility`, etc.) are non-clustered (secondary indexes).

Q44: Explain the query execution order in SQL.

SQL does not execute in the order it is written. The actual execution order is:

1. **FROM** — Identify source tables and perform JOINs
2. **WHERE** — Filter individual rows
3. **GROUP BY** — Group remaining rows
4. **HAVING** — Filter groups
5. **SELECT** — Evaluate expressions and compute columns
6. **DISTINCT** — Remove duplicate rows
7. **ORDER BY** — Sort the result
8. **LIMIT / OFFSET** — Return subset of rows

This is why you **cannot** use a column alias defined in SELECT inside a WHERE clause (WHERE executes before SELECT), but you **can** use it in ORDER BY.

Q45: What is a deadlock? How can it occur in your system?

A **deadlock** occurs when two or more transactions wait for each other to release locks, creating a circular dependency.

Scenario in our project:

1. Transaction A locks Slot 5 (to park Vehicle 1)
2. Transaction B locks Slot 8 (to park Vehicle 2)
3. Transaction A now needs Slot 8 (to transfer Vehicle 1) — waits for B
4. Transaction B now needs Slot 5 (to transfer Vehicle 2) — waits for A
5. **Deadlock!** Neither can proceed.

MySQL's solution: InnoDB automatically detects deadlocks and rolls back the transaction with the fewest changes.

Prevention strategies:

- Always acquire locks in the same order (e.g., lower slot_id first)
- Keep transactions short
- Use appropriate isolation levels

Quick Reference: All 50 Queries at a Glance

#	Category	Description
1	SELECT	List all facilities
2	JOIN	Available slots per facility
3	WHERE	Premium/VIP users
4	JOIN	Vehicles with owner details
5	JOIN	Currently parked vehicles
6	GROUP BY	Slot count per facility
7	GROUP BY	Revenue per facility
8	AVG	Average parking duration

9	GROUP BY	Revenue by payment method
10	GROUP BY	Vehicle count per user
11	INNER JOIN	Users with payments
12	LEFT JOIN	All users with/without reservations
13	RIGHT JOIN	All slots with/without records
14	CROSS JOIN	All user-facility combinations
15	SELF JOIN	Staff at same facility
16	Subquery	Above-average spenders
17	Subquery	Highest occupancy facility
18	NOT IN	Never-used slots
19	Correlated	Users spending > 500
20	UNION	Users with reservations or payments
21	INTERSECT	Users with both
22	INSERT	Add new user
23	UPDATE	Mark slot occupied
24	UPDATE	Change membership
25	DELETE	Remove cancelled reservations
26	VIEW	Facility occupancy dashboard
27	VIEW	Revenue summary
28	VIEW	Active sessions with estimated bill
29	PROCEDURE	Park vehicle
30	PROCEDURE	Exit vehicle and bill
31	PROCEDURE	Facility report
32	TRIGGER	Auto-occupy on park
33	TRIGGER	Auto-free on exit
34	TRIGGER	Prevent user delete
35	WINDOW	Rank facilities by revenue
36	WINDOW	Running total of payments
37	WINDOW	Rank users by spending
38	TCL	Atomic slot transfer
39	TCL	SAVEPOINT example
40	DCL	GRANT/REVOKE examples
41	HOUR()	Peak parking hours
42	DATE_FORMAT	Monthly revenue trend
43	LEFT JOIN	Slot utilization count
44	GROUP BY	Staff salary report
45	CASE	EV slot availability
46	GROUP_CONCAT	Multi-vehicle users
47	ORDER BY	Longest parking session
48	GROUP BY	Revenue per slot type
49	CASE	Membership discount simulation
50	EXISTS	Facilities with EV slots